LAPORAN
WORKSHOP Mobile Web Server
(TIF330805)
Acara 19



Nama Tugas
POSTMAN DAN API AUTHENTICATION

Nama:
Adam Ridho Saifulloh

NIM:
E41250216

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)
TEKNIK INFORMATIKA KAB. NGANJUK
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI JEMBER
TAHUN 2026

A. PROSEDUR KERJA

Prosedur kerja Acara 19 menambahkan resource baru berupa autentikasi (registrasi user) ke dalam arsitektur MVC yang sudah berjalan pada acara sebelumnya. Penambahan meliputi tabel users, Model User, Controller AuthController, dan pemetaan baru pada Router, kemudian seluruhnya diuji menggunakan Postman.

Langkah 1 — Membuat Postman Collection dengan URL Baru

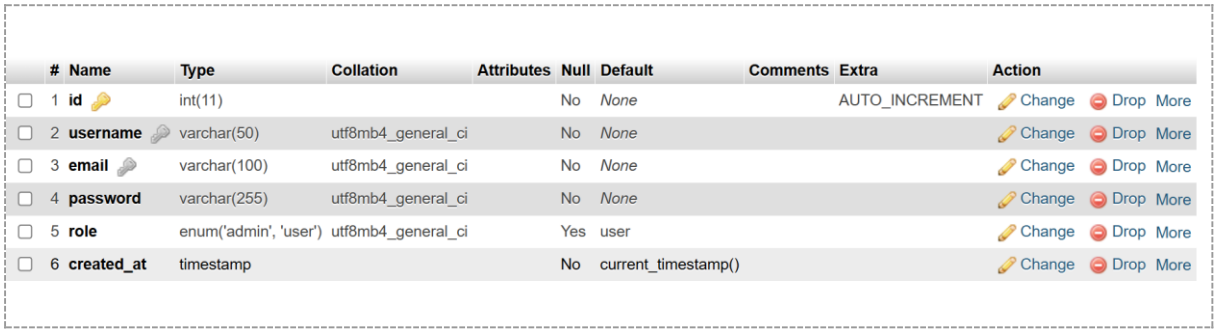
1. Membuat Collection baru bernama "API Sistem Informasi Akademik (MVC)".
2. Membuat Environment "Local" dengan variabel base_url bernilai http://localhost/sistem-informasi-akademik/public. Endpoint kini tanpa akhiran .php karena diarahkan oleh Front Controller dan Router.
3. Memindahkan request mahasiswa dari Acara 17–18 ke Collection ini dengan URL {{base_url}}/mahasiswa. Request pada Collection dikelompokkan menjadi folder Mahasiswa (berhasil dan error), Buku, dan Autentikasi.

Langkah 2 — Membuat Tabel users

Tabel users ditambahkan pada berkas database/db_workshop.sql. Kolom username dan email dibuat UNIQUE agar tidak ada data ganda, kolom password bertipe VARCHAR(255) untuk menampung hash, dan kolom role bernilai default user.

File: database/db_workshop.sql (tambahan)

```sql
CREATE TABLE IF NOT EXISTS users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  email VARCHAR(100) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  role ENUM('admin', 'user') DEFAULT 'user',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```



#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	username	varchar(50)	utf8mb4_general_ci		No	None			Change Drop More
3	email	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
4	password	varchar(255)	utf8mb4_general_ci		No	None			Change Drop More
5	role	enum('admin', 'user')	utf8mb4_general_ci		Yes	user			Change Drop More
6	created_at	timestamp			No	current_timestamp()			Change Drop More

Gambar 2. Struktur tabel users

Langkah 3 — Membuat Model User

Model User menangani akses data tabel users melalui PDO prepared statement. Method create() menyimpan user baru, sedangkan findByUsername() dipakai untuk memeriksa apakah username sudah terdaftar.

File: app/models/User.php

```php
<?php

class User
{
    private PDO $db;

    public function __construct()
    {
        $this->db = Database::getConnection();
    }

    public function create(array $data): string
    {
        $stmt = $this->db->prepare(
            'INSERT INTO users (username, email, password)
             VALUES (:u, :e, :p)'
        );

        $stmt->execute([
            ':u' => $data['username'],
            ':e' => $data['email'],
            ':p' => $data['password'],
        ]);

        return $this->db->lastInsertId();
    }

    public function findByUsername(string $username)
    {
        $stmt = $this->db->prepare(
            'SELECT * FROM users WHERE username = :u'
        );

        $stmt->execute([
            ':u' => $username,
        ]);

        return $stmt->fetch();
    }
}
```

Langkah 4 — Membuat Controller AuthController

AuthController merupakan Controller pertama yang tidak mengikuti pola CRUD standar (index/store/update/destroy) karena hanya berisi aksi tunggal register(). Kata sandi tidak disimpan sebagai teks biasa, melainkan di-hash menggunakan password_hash() dengan algoritma PASSWORD_DEFAULT. Seluruh response dikirim melalui Response::json() dari Acara 18.

File: app/controllers/AuthController.php

```php
<?php

class AuthController
{
    private User $model;

    public function __construct()
    {
        $this->model = new User();
    }

    public function register(): void
    {
        $data = json_decode(file_get_contents('php://input'), true);

        if (!is_array($data)) {
            Response::json(400, false, 'Body request harus berupa JSON yang
valid');
        }

        if (
            empty($data['username']) ||
            empty($data['email']) ||
            empty($data['password'])
        ) {
            Response::json(
                400,
                false,
                'Username, email, dan password wajib diisi'
            );
        }

        if (!filter_var($data['email'], FILTER_VALIDATE_EMAIL)) {
            Response::json(422, false, 'Format email tidak valid', null, [
                'email' => 'Masukkan alamat email yang valid.',
            ]);
        }

        if ($this->model->findByUsername(trim($data['username']))) {
            Response::json(409, false, 'Username sudah digunakan');
```

```
        }

        $data['username'] = trim($data['username']);
        $data['email'] = trim($data['email']);
        $data['password'] = password_hash(
            $data['password'],
            PASSWORD_DEFAULT
        );

        try {
            $id = $this->model->create($data);
        } catch (PDOException $e) {
            // Constraint UNIQUE juga menjaga agar email tidak boleh duplikat.
            if ($e->getCode() === '23000') {
                Response::json(409, false, 'Username atau email sudah
digunakan');
            }
            throw $e;
        }

        Response::json(
            201,
            true,
            'Registrasi berhasil',
            ['id' => $id]
        );
    }
}
```

Alur pada method register(): (1) body JSON dibaca dan diperiksa validitasnya (400); (2) field username, email, dan password diperiksa kelengkapannya (400); (3) username diperiksa agar belum terpakai (409); (4) password di-hash lalu data disimpan; (5) apabila constraint UNIQUE pada email atau username dilanggar, PDOException dengan kode 23000 ditangkap dan dikembalikan sebagai 409; (6) jika berhasil dikembalikan kode 201.

Langkah 5 — Mendaftarkan Resource register pada Router

Karena register bukan resource CRUD biasa, ditambahkan satu entri pada peta resource dan penanganan khusus pada Router.php sebelum blok switch method HTTP. Endpoint register hanya menerima method POST; method lain dijawab dengan kode 405 beserta header Allow.

File: app/core/Router.php (bagian yang ditambahkan)

```
 private array $map = [
        'mahasiswa' => 'MahasiswaController',
        'buku'      => 'BukuController',   // Tugas Praktik Mandiri 1
        'register'  => 'AuthController',
    ];
```

```php
    public function dispatch(): void {
        $uri = trim((string) parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH),
'/');
        $segments = explode('/', $uri);
        $resource = end($segments) ?: 'mahasiswa';

        $controllerName = $this->map[$resource] ?? null;
        if (!$controllerName) {
            $this->error(404, 'Endpoint tidak ditemukan');
        }

        $controller = new $controllerName();

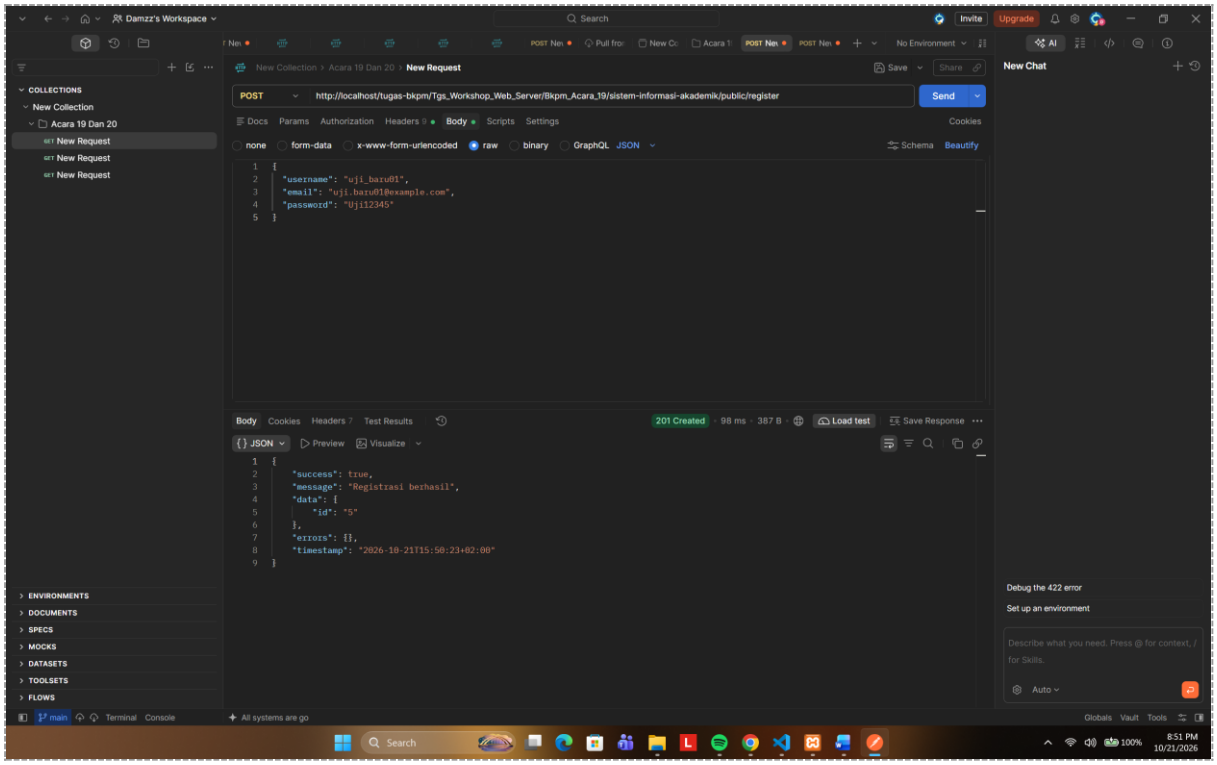
        // Endpoint registrasi bukan bagian dari CRUD standar.
        if ($resource === 'register') {
            if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
                header('Allow: POST');
                Response::json(405, false, 'Method tidak diizinkan');
            }
            $controller->register();
            return;
        }
```

Langkah 6 — Menguji dan Mendokumentasikan pada Collection

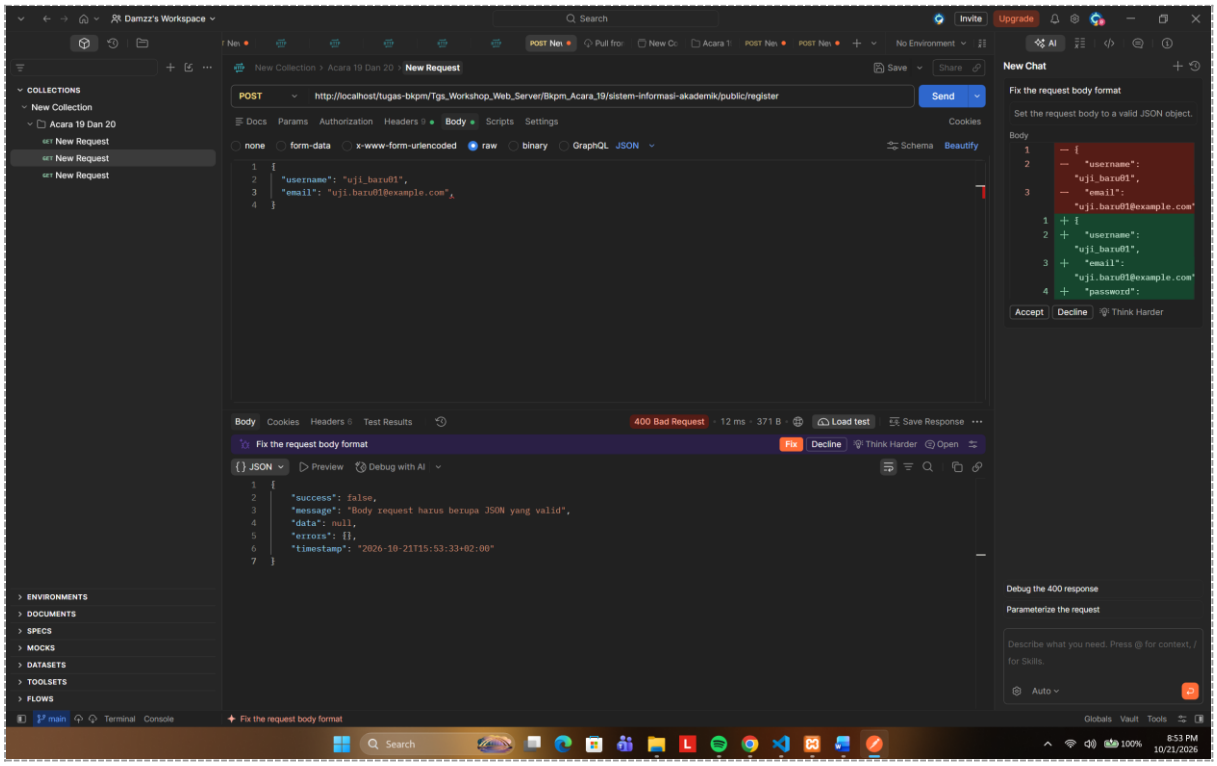
Request baru POST {{base_url}}/register ditambahkan ke folder "D. Autentikasi (Acara 19)" pada Collection, dengan header Content-Type: application/json. Tiga skenario diuji seperti pada Tabel 1; setiap request dilengkapi script test Postman untuk memeriksa kode status dan format JSON.

No	Skenario	Body request	Status	Hasil
1	Registrasi berhasil	username: adam_test, email: adam_test@example.com, password: Rahasia123!	201	Registrasi berhasil
2	Field tidak lengkap	username dan email saja, password tidak dikirim	400	Field wajib diisi
3	Email tidak valid	email: bukan-email	422	Format email tidak valid

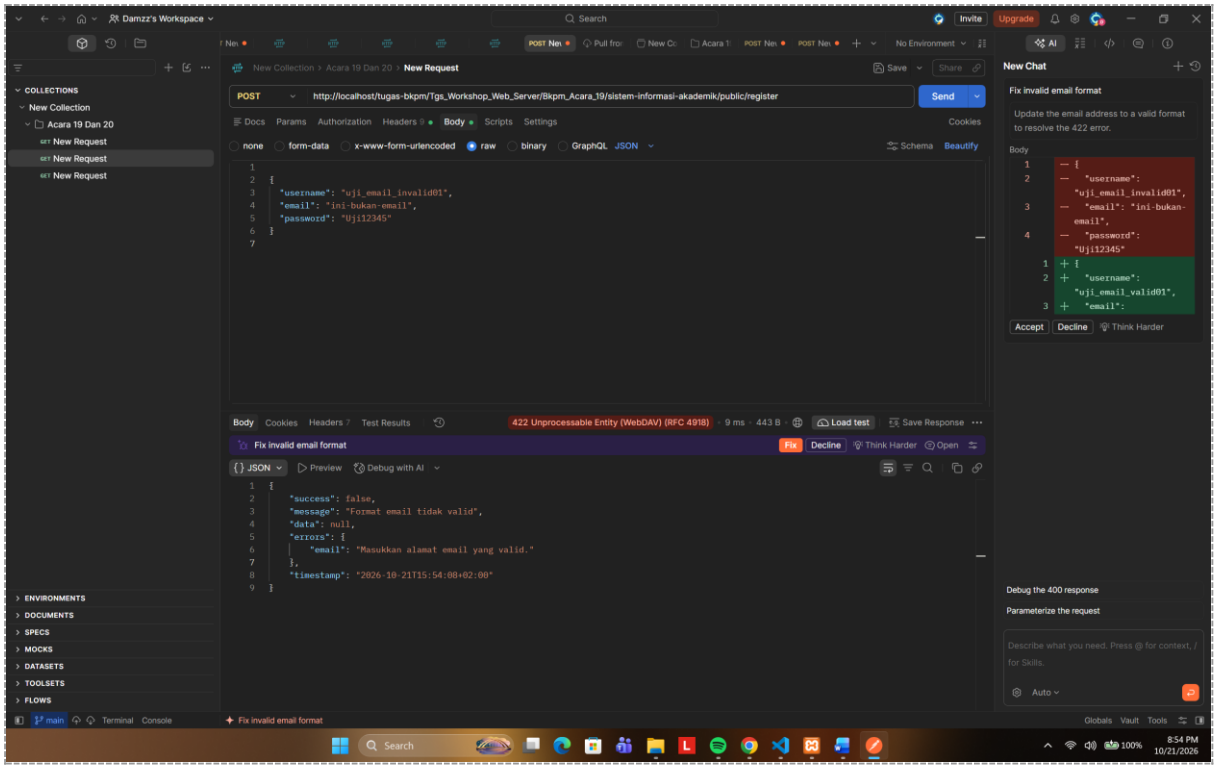
Tabel 1. Skenario pengujian endpoint register



Gambar 3. Pengujian registrasi berhasil



Gambar 4. Pengujian registrasi dengan field tidak lengkap



Gambar 5. Pengujian registrasi dengan email tidak valid

Contoh response sukses dan gagal disimpan pada masing-masing request sebagai bahan dokumentasi awal.

B. TUGAS PRAKTIK MANDIRI

Tugas 1: Validasi Email dengan filter_var()

Validasi format email ditambahkan menggunakan filter_var() dengan filter FILTER_VALIDATE_EMAIL sebelum data disimpan. Pada implementasi ini validasi ditempatkan pada AuthController::register() tepat sebelum data diteruskan ke Model User, sehingga data dengan email tidak valid tidak pernah sampai ke lapisan database. Jika tidak valid, server mengembalikan kode 422 beserta field errors yang menyebutkan field email (field errors berasal dari Tugas Acara 18).

File: app/controllers/AuthController.php (bagian yang ditambahkan)

```
if (!filter_var($data['email'], FILTER_VALIDATE_EMAIL)) {
            Response::json(422, false, 'Format email tidak valid', null, [
                'email' => 'Masukkan alamat email yang valid.',
            ]);
        }
```

Pengujian dilakukan dengan request nomor 3 pada Tabel 1 (email “bukan-email”). Hasil yang diharapkan adalah status 422 dengan errors.email berisi pesan “Masukkan alamat email yang valid.”

Tugas 2: Environment Kedua untuk Simulasi Staging

Environment kedua bernama “Staging” dibuat pada Postman untuk mensimulasikan lingkungan staging. Environment ini memiliki variabel base_url yang sama namun bernilai berbeda dari Local, sehingga seluruh request pada Collection yang memakai {{base_url}} dapat dijalankan pada lingkungan lain hanya dengan mengganti Environment yang dipilih, tanpa mengubah satu pun request.

Environment	Variabel	Nilai
Local	base_url	http://localhost/sistem-informasi-akademik/public
Staging	base_url	http://staging.localhost/sistem-informasi-akademik/public

Tabel 2. Perbandingan Environment Local dan Staging